

Module 1

IEEE 1364-1995 (Verilog-95)

Learning objectives

- Master the first standardized Verilog language (IEEE 1364-1995) and its core syntax for RTL design...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["IEEE 1364-1995 Context"]

T2["Modules, Ports, and Hierarch"]

T1 --> T2

T3["Nets and Variables: wire and"]

T2 --> T3

Master the first standardized Verilog language (IEEE 1364-1995) and its core syntax for RTL design and simulation.

Overview

- This module establishes the foundation for the Verilog/SystemVerilog version-centric course. You'll...

How to learn this module

Suggested learning path (1/2)

- Skim this document — Goal, Overview, and Topics
Covered set the IEEE scope for Module 1.
- Study design architecture — See how DUTs, examples, and testbenches fit together under ``module1/``.
- Work through labs in order — Open ``module1/EXAMPLES.md`` and run each ``make clean && make run`` from...
- Run the full module script — ``./scripts/module1.sh`` from the repo root to simulate all examples and...
- Complete the exercises — Try each exercise in this document before reading solutions or peeking at...

Suggested learning path (2/2)

- Review Common Pitfalls — Can you explain each mistake and the fix?
- Self-check — Use ``module1/CHECKLIST.md`` before starting the next module.

Syllabus topics

1. IEEE 1364-1995 Context (1/3)

- Modules and hierarchy: Module declaration, port list, instantiation
- Nets: ``wire`` (and ``wand``, ``wor``, ``tri``; rarely used in RTL)
- Variables: ``reg`` for procedural assignment outputs
- Continuous assignment: ``assign`` for combinational logic
- Procedural blocks: ``always``, ``initial`` with explicit sensitivity lists
- Delays and timing: ``#``, ``@``, ``wait``

1. IEEE 1364-1995 Context (2/3)

- Tasks and functions: Non-ANSI style (declarations separate from port list)
- System tasks: ``$display``, ``$monitor``, ``$finish``, ``$stop``, ``$time``
- No ANSI-style port declarations (added in 1364-2001)
- No `**`always @*`**` (added in 1364-2001)
- No generate (added in 1364-2001)
- No signed keyword, no multi-dimensional arrays (1364-2001)

1. IEEE 1364-1995 Context (3/3)

- No SystemVerilog: no ``logic``, ``always_comb``, ``always_ff``, interfaces, packages

2. Modules, Ports, and Hierarchy (1364-1995)

- Port list: Only port names: `(a, b, y)`.
- Port direction: Declared inside the module: `input`, `output`.
- Port type: Declared inside the module: `wire` or `reg` (default for `input` is `wire`; for `output`...
- Named port connection: `.port\_name(signal\_name)` — recommended for readability.
- Positional: Order must match port list.

3. Nets and Variables: wire and reg (1/2)

- Represents a physical connection between components.
- Driven by continuous assignment (``assign``) or by module output.
- Cannot be assigned inside ``always`` or ``initial`` (use ``reg`` for that).
- Holds a value; can be assigned in ``always`` or ``initial``.
- Does not imply a flip-flop; flip-flops come from ``always @(posedge clk)`` (or similar).
- Used for outputs that are driven procedurally.

3. Nets and Variables: wire and reg (2/2)

- Both ``wire`` and ``reg`` are 4-state: ``0``, ``1``, ``X`` (unknown), ``Z`` (high-impedance).

4. Continuous Assignment (assign)

- Right-hand side: Any expression of ``wire``/``reg`` and operators.
- Left-hand side: Must be a ``wire`` (or similar net).

5. Procedural Blocks: always and initial (1364-1995)

- Repeated execution; sensitivity list defines when it runs.
- 1364-1995: Sensitivity list is mandatory and explicit — no ``@*``.
- Use blocking ``=`` for combinational, nonblocking ``<=`` for sequential (flip-flops).
- Example (D flip-flop):
``module1/examples/sequential_dff/dff.v``
- Runs once at time zero. Used for testbenches:
stimulus, ``$display``, ``$finish``.

6. Delays and Timing

- ``#n``: Delay by ``n`` time units (e.g. ``#10``).
- ``@(event)``: Wait for event (e.g. ``@(posedge clk)``).
- ``wait(condition)``: Wait until condition is true.

7. Tasks and Functions (1995 Style)

- Function: Returns a single value; no delays (zero-time).
- Task: Can have delays and multiple outputs (output/inout args).

8. Simple Testbench (1364-1995)

- ``$display``: Print once.
- ``$monitor``: Print whenever a listed signal changes (optional).
- ``$finish``: End simulation.

Hands-on examples

Module 1 toolchain check

```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog  
-V 2>&1 | head -1)"
```

Terminal: module1_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Example 1: Modules and Ports

- 1995-style module declaration and port list
- Named and positional instantiation
- Non-ANSI ports, separate direction/type declarations

Demo: Modules and Ports

```
$ cd module1/examples/modules_ports && make clean && make run
```

```
Terminal: ex01_modules_ports  
rm -f top  
iverilog -o top and_gate.v  
vvp top  
a=0 b=0 y=0  
a=0 b=1 y=0  
a=1 b=0 y=0  
a=1 b=1 y=1  
and_gate.v:30: $finish called at 40 (1s)
```


Example 2: Nets and Variables

- wire vs reg, vectors, 4-state values
- When to use wire vs reg, single driver per net

Demo: Nets and Variables

```
$ cd module1/examples/nets_variables && make clean && make run
```

```
Terminal: ex02_nets_variables  
rm -f data_types  
iverilog -o data_types data_types.v  
vvp data_types  
w=1 bus=11111111 r=1 counter= 42  
data_types.v:22: $finish called at 30 (1s)
```

Example 3: Continuous Assignment

- assign for combinational logic (2:1 mux)
- Left-hand side must be net; expression on right

Demo: Continuous Assignment

```
$ cd module1/examples/continuous_assign && make clean && make run
```

Terminal: ex03_continuous_assign

```
rm -f top
iverilog -o top mux2.v
vvp top
sel a b | y
0 0 0 | 0
0 0 1 | 0
0 1 0 | 1
0 1 1 | 1
1 0 0 | 0
1 0 1 | 1
1 1 0 | 0
1 1 1 | 1
mux2.v:34: $finish called at 80 (1s)
```

Example 4: Continuous Assignment Gates

- XOR, NAND, NOR with assign
- Same as above; multiple gates in one example

Demo: Continuous Assignment Gates

```
$ cd module1/examples/continuous_assign_gates && make clean && make  
run
```

Terminal: ex04_continuous_assign_gates

```
rm -f top  
iverilog -o top gates.v  
vvp top  
a b | xor nand nor  
0 0 | 0 1 1  
0 1 | 1 1 0  
1 0 | 1 1 0  
1 1 | 0 0 0  
gates.v:46: $finish called at 40 (1s)
```

Example 5: Adder

- Half adder and full adder with assign (sum and carry)
- Multi-output combinational logic; wire for all outputs

Demo: Adder

```
$ cd module1/examples/adder && make clean && make run
```

Terminal: ex05_adder

```
rm -f top
iverilog -o top adder.v
vvp top
Half adder: a b | sum cout
  0 0 | 0 0
  0 1 | 1 0
  1 0 | 1 0
  1 1 | 0 1
Full adder: a b cin | sum cout
  0 0 0 | 0 0
  1 0 0 | 1 0
  0 1 0 | 1 0
  1 1 0 | 0 1
  1 0 1 | 0 1
  1 1 1 | 1 1
adder.v:49: $finish called at 100 (1s)
```


Example 6: Procedural Blocks

- always @(inputs) and always @(posedge clk) with explicit sensitivity
- initial for testbench; 1995-style task/function
- No `always @\*`; blocking vs nonblocking

Demo: Procedural Blocks

```
$ cd module1/examples/procedural && make clean && make run
```

Terminal: ex06_procedural

```
rm -f always_initial task_function_1995
iverilog -o always_initial always_initial.v
vvp always_initial
y_comb=1 q=1
always_initial.v:35: $finish called at 40 (1s)
iverilog -o task_function_1995 task_function_1995.v
vvp task_function_1995
add8(10,20) = 30
rst_n after task = 1
task_function_1995.v:41: $finish called at 120 (1s)
```

Example 7: Sequential D Flip-Flop

- D flip-flop with async reset: always @(posedge clk or negedge rst_n) and <=
- Edge-sensitive always; nonblocking for sequential logic

Demo: Sequential D Flip-Flop

```
$ cd module1/examples/sequential_dff && make clean && make run
```

Terminal: ex07_sequential_dff

```
rm -f top
iverilog -o top dff.v
vvp top
D flip-flop (1364-1995): always @(posedge clk or negedge rst_n); q <= d
  t=22 d=1 q=1
  t=32 d=0 q=0
  t=42 d=1 q=1
dff.v:46: $finish called at 62 (1s)
```

Example 8: Delays and Timing

- `#n`, `@(posedge clk)`, `wait(condition)`; clock generation with `forever`
- Delay-based and event-based timing in testbenches

Demo: Delays and Timing

```
$ cd module1/examples/delays_timing && make clean && make run
```

Terminal: ex08_delays_timing

```
rm -f top
iverilog -o top timing.v
vvp top
Delays and timing (1364-1995): #, @(posedge clk), wait(condition)
t  clk rst_n start count ready
t=55 clk=1 rst_n=1 count= 1
t=65 clk=1 rst_n=1 count= 2
t=75 clk=1 rst_n=1 count= 3
t=85 clk=1 rst_n=1 count= 4
timing.v:47: $finish called at 135 (1s)
```

Example 9: Testbenches

- Minimal testbench with initial, #, \$display, \$finish
- Stimulus generation, termination with ` \$finish `

Demo: Testbenches

```
$ cd module1/examples/testbenches && make clean && make run
```

Terminal: ex09_testbenches

```
rm -f test_and_gate
iverilog -o test_and_gate -I ../dut/simple_gates ../dut/simple_gates/and_gate.v test_and_g
vvp test_and_gate
a b y
0 0 0
0 1 0
1 0 0
1 1 1
test_and_gate.v:19: $finish called at 40 (1s)
```


Common pitfalls

Incomplete Sensitivity List

- Mistake: ``always @(sel) y = sel ? b : a;`` (missing ``a`` and ``b``)
- Reality: In 1364-1995 you must list every input that the block reads
- Correct: ``always @(a or b or sel) y = sel ? b : a;``
- Why: Incomplete lists cause simulation/synthesis mismatches and inferred latches

Port Type Mismatch

- Mistake: Declaring as ``wire`` an output that is assigned in ``always``
- Reality: Outputs driven in procedural blocks must be ``reg``
- Correct: ``output y; reg y;`` when ``y`` is assigned in ``always``
- Why: ``wire`` cannot be the target of procedural assignment

Simulation Never Ends

- Mistake: Testbench without ``$finish``
- Correct: Always call ``$finish`` after stimulus (e.g. ``#1000; $finish;``)
- Why: Simulator runs until explicit termination or limit

Using 2001/SystemVerilog Constructs in 1995

- Mistake: Writing ``always @*`` or ANSI ports and assuming 1995
- Reality: ``always @*`` and ANSI ports are 1364-2001;
``logic`/`always_comb`` are SystemVerilog
- Correct: For strict 1995, use explicit sensitivity and non-ANSI ports
- Why: Keeps code portable to 1995-only tools and clarifies version scope

Confusing reg with Registers

- Mistake: Thinking ``reg`` creates a flip-flop
- Reality: ``reg`` is a variable type; flip-flops come from ``always @(posedge clk)`` with ``<=``
- Correct: Use ``reg`` for any procedural output; use clocked ``always`` for sequential logic
- Why: Clarifies design intent and avoids synthesis surprises

Practice & assessment

What you should know (1/2)

- ✓ Write modules in IEEE 1364-1995 style (non-ANSI ports, explicit sensitivity)
- ✓ Use ``wire`` and ``reg`` correctly for nets and procedural outputs
- ✓ Describe combinational logic with ``assign`` and ``always @(inputs)``
- ✓ Describe sequential logic with ``always @(posedge clk)`` and nonblocking assignment
- ✓ Use ``initial``, ``#``, ``@``, and ``wait`` for testbenches
- ✓ Use 1364-1995 tasks and functions (non-ANSI form)

What you should know (2/2)

- ✓ State what 1364-1995 does not include (ANSI, `@\*`, generate, SystemVerilog)

Exercises

- Module and Ports (1995)
- wire vs reg
- Simple Testbench
- Limitations
- Incomplete Sensitivity List

Exercises

- Port Type Mismatch
- Simulation Never Ends
- Using 2001/SystemVerilog Constructs in 1995
- Confusing reg with Registers

Summary & next steps

- Master the first standardized Verilog language (IEEE 1364-1995) and its core syntax for RTL design...
- Complete module1/CHECKLIST.md
- Review module1/EXAMPLES.md and run each lab
- Next: Next module in course